

Full-Stack Web Development of an Automated Assessment Platform for UML Diagrams

Wang Yu Teng

U2122796L

Supervisor: Dr. Loke Yuan Ren

Examiner: Prof. Zhang Hanwang

● Outline

01

Background

Problem · Prior work · Objectives ·
Contributions

02

Demo

Pre-recorded walkthrough of the platform

03

Implementation

Architecture · Lifecycle · AI pipeline ·
Deployment

04

Experiment

RQs · Setup · Metrics · Results · Threats

05

Conclusion

Summary & future work

01

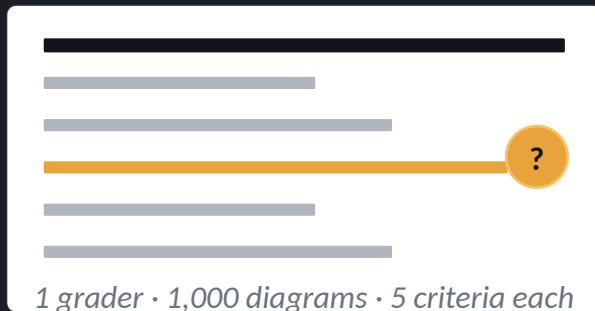
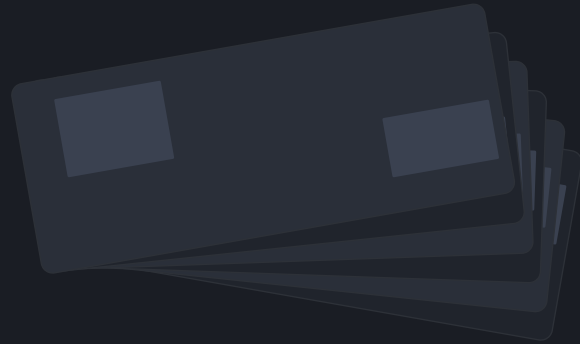
SECTION

Background



• The Grading Bottleneck

Per-cohort grading effort



~5 min

per diagram (conservative pace)

1,000

diagrams · 200 students × 5 each

≈83 hrs

of TA effort per cohort

Inconsistency

different graders interpret rubrics differently
— a recognised threat to assessment validity

• Prior Work & The Research Gap

Traditional methods

- UMLGrader — heuristic, IBM Rational Rose only
- Bian et al. — Levenshtein + WordNet, class-only
- Modi / Jebli — XMI parsing, Modelio-locked

✗ Tied to specific tools, class-only,
no semantic richness

LLM-based methods

- Wang et al. — GPT-4o on 40 students, 1.18-pt gap, weak on class/sequence
- Bouali et al. — GPT-o1-mini + Sonnet + Llama, ICC = 0.76
- DUET — GPT-4o + Mistral-7B, formative only

⚠ Earlier-generation models, weaker reasoning,
format-compliance issues

The gap: No study has revisited LLM-based UML grading using SOTA models with API-level structured-output enforcement — and no production-grade assessment platform embeds such a pipeline.

• Project Objectives

01 [O1] Full-stack platform

End-to-end assessment lifecycle from course setup to grade publication.

02 [O2] LLM grading pipeline

Structured-output enforcement, schema validation, score clamping.

03 [O3] Reduce TA effort

Streamlined two-panel review UI with accept/reject workflow.

04 [O4] Empirical evaluation

Compare SOTA LLMs across accuracy, cost, consistency.

05 [O5] Containerised deployment

Docker + Kubernetes for on-premise NTU infrastructure.

• My Contributions

01 Full-stack platform

React 19 SPA + Hono backend + Drizzle/PostgreSQL + Graphile Worker. 16 tables across 5 functional groups; 130+ endpoints across 11 route modules.

02 AI grading pipeline

Provider-agnostic factory (OpenAI / Anthropic in production; Gemini via experiment harness), hybrid symbolic + neural grading, Zod-validated structured output with score clamping, human-in-the-loop accept/reject.

03 Custom UML editor & viewer

React Flow with custom UML node/edge types. Structured JSON $\leftrightarrow$ PlantUML DSL — the editor's typed shape enables a deterministic graph-diff against the reference answer.

04 Empirical LLM evaluation

6 models $\times$ 32 submissions $\times$ 5 runs = 960 API calls. Weighted aggregate across accuracy, consistency, cost, feedback, speed, rubric adherence.

05 Containerised deployment

Multi-stage Docker on Node 20 Alpine + Kubernetes manifests with HPA on web tier, single-replica worker.

02

SECTION

Demo



• Demo

Course management & CSV roster •

Question pool with filtering •

Assignment builder • timing + focus •

Two-panel grading interface •

Grade override with audit trail •

• Student attempt • auto-save 30s

• Focus monitoring • auto-submit

• AI grading: written + UML

• Accept / reject AI suggestions

• Cost & token tracking dashboard



Demo video plays here • ~3 min

openscreen-edited • annotations + highlights

Staff workflow (60s) → **Student workflow** (60s) → **Grading loop** (60s)

03

SECTION

Implementation



• System Architecture

01 Monolithic, not microservices

Single codebase, modular route files; one Docker image, one deployment unit.

02 Web ↔ worker separation

LLM calls take 5–60s; decoupling keeps the request path responsive during exam-period spikes.

03 PostgreSQL job queue

Graphile Worker — no Redis, transactional with application data.

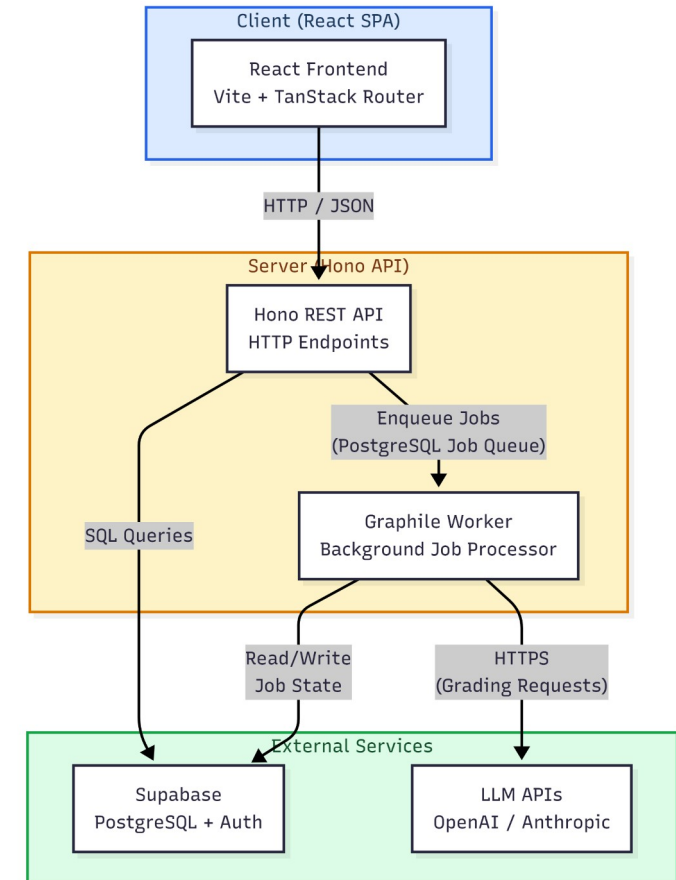


Figure 3.1 — Three-tier architecture

• Design for Trust & Consistency

01 Defence-in-depth — JWT → RBAC → RLS

Three independent gates per request. JWT verified at the edge → `requireRole()` enforces role and course scope → every query runs in a Postgres transaction switched to the authenticated role with row-level policies. A forgotten auth check in one layer still cannot leak another student's data.

02 One image, two roles

The same multi-stage Docker image starts the web server by default, or the Graphile worker via a CMD override. Web tier scales horizontally on HPA for exam bursts; worker stays at a single replica to avoid LLM rate-limit contention. Same code, same migrations — no drift between tiers.

03 Stateless per-job grading

Each grading job opens a fresh `generateText()` call — no message history, no app-level prompt or response cache, temperature 0.0 where supported. Submission B never sees A's prompt or output, so identical inputs always produce identical scoring — reproducibility is structural, not policy.

● Tech Stack

Language & Runtime | TypeScript 5.9 · Node.js 20

Frontend | React 19 · Vite · TanStack Router · TanStack Query · Tailwind CSS · React Flow (xyflow 12.10)

Backend | Hono 4.11 · Drizzle ORM · Zod 4.3 · Graphile Worker 0.16

Auth | Supabase Auth · Custom JWT (dual-validation)

AI | Vercel AI SDK 6.0 → OpenAI / Anthropic

UML | plantuml-encoder

Testing | Vitest · Playwright

DevOps | Docker (multi-stage Node 20 Alpine) · Kubernetes / k3s · GitHub Actions

Database Design

01 JSONB polymorphic content

Three question types (MCQ / written / UML) without messy joins.

02 Postgres enums for state

submission_status: draft → submitted → late → grading → graded, enforced at DB level.

03 Composite uniqueness

Guarantees integrity under concurrent exam load — race conditions cannot create duplicates.

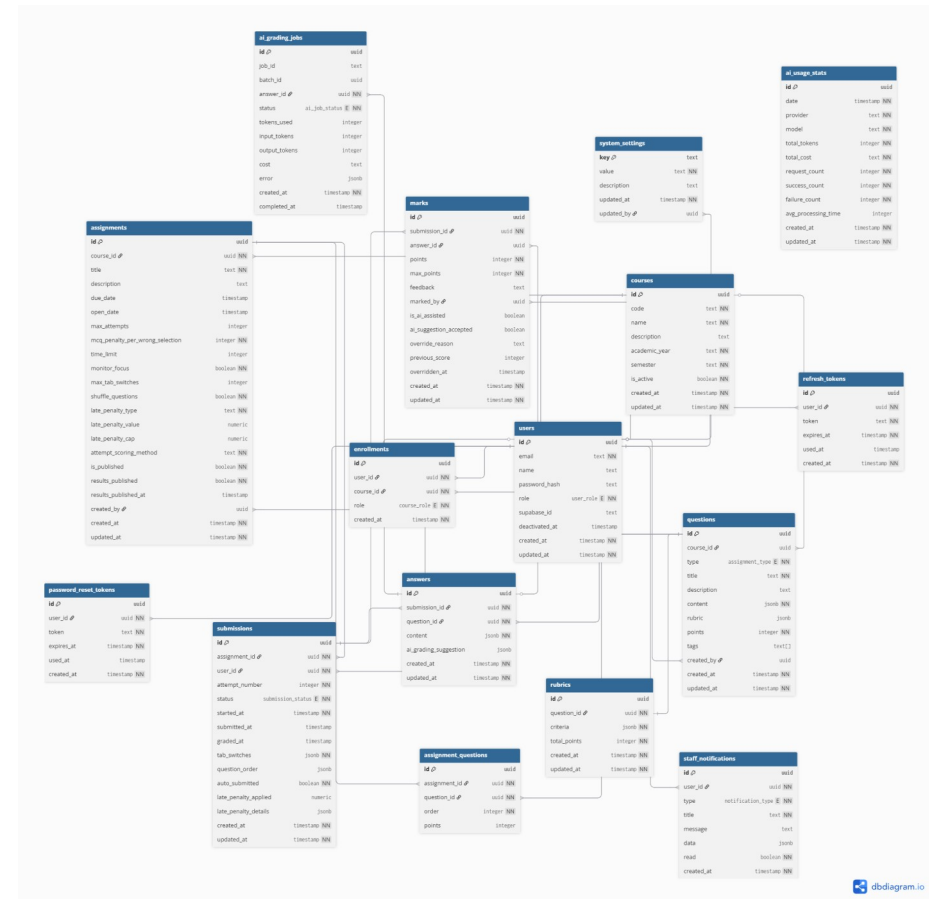
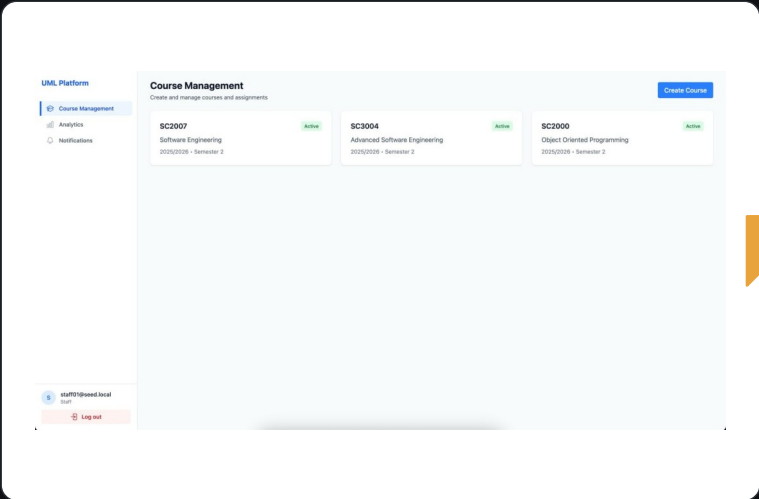


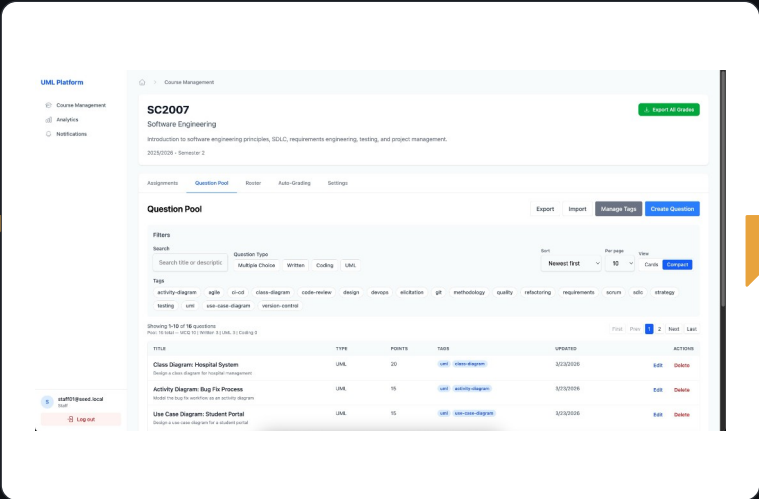
Figure 3.2 — Entity-Relationship Diagram (16 tables, 5 functional groups)

● Lifecycle • Staff Side



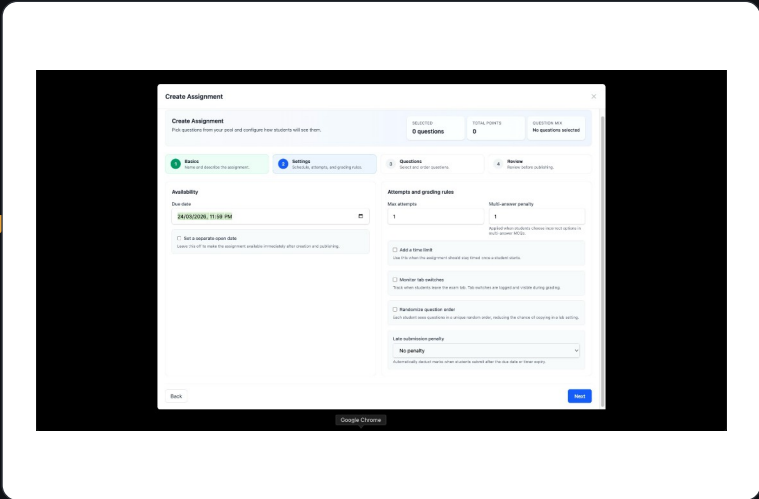
Course Management

Individual + bulk CSV enrollment with per-row status reporting.



Question Pool

Course-scoped, tag-filterable, reusable across assignments.



Assignment Builder

Due date • time limit • attempts • late penalty • shuffle • focus monitor.

• Lifecycle • Student Side

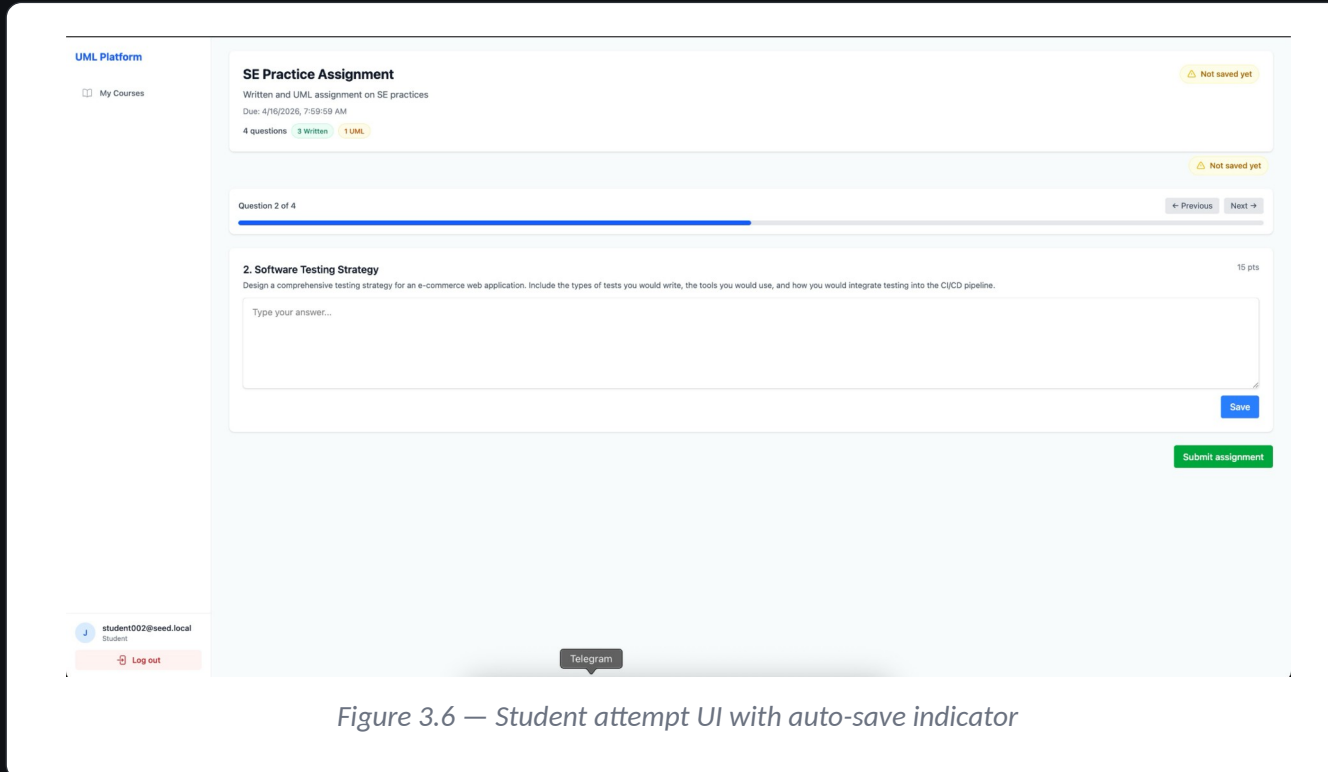


Figure 3.6 — Student attempt UI with auto-save indicator

01 Idempotent start

If a draft exists, return it; never duplicate.

02 Auto-save every 30s

Visible timestamp indicator on changes.

03 Dual-layer timer

Client countdown for UX + server cron polling every 60s for expired drafts.

04 Opt-in focus monitoring

Tab switches counted; threshold breach → auto-submit current state.

05 Atomic submission

Entire submission pipeline wrapped in one DB transaction.

Submission state machine



• Custom UML Diagram Editor

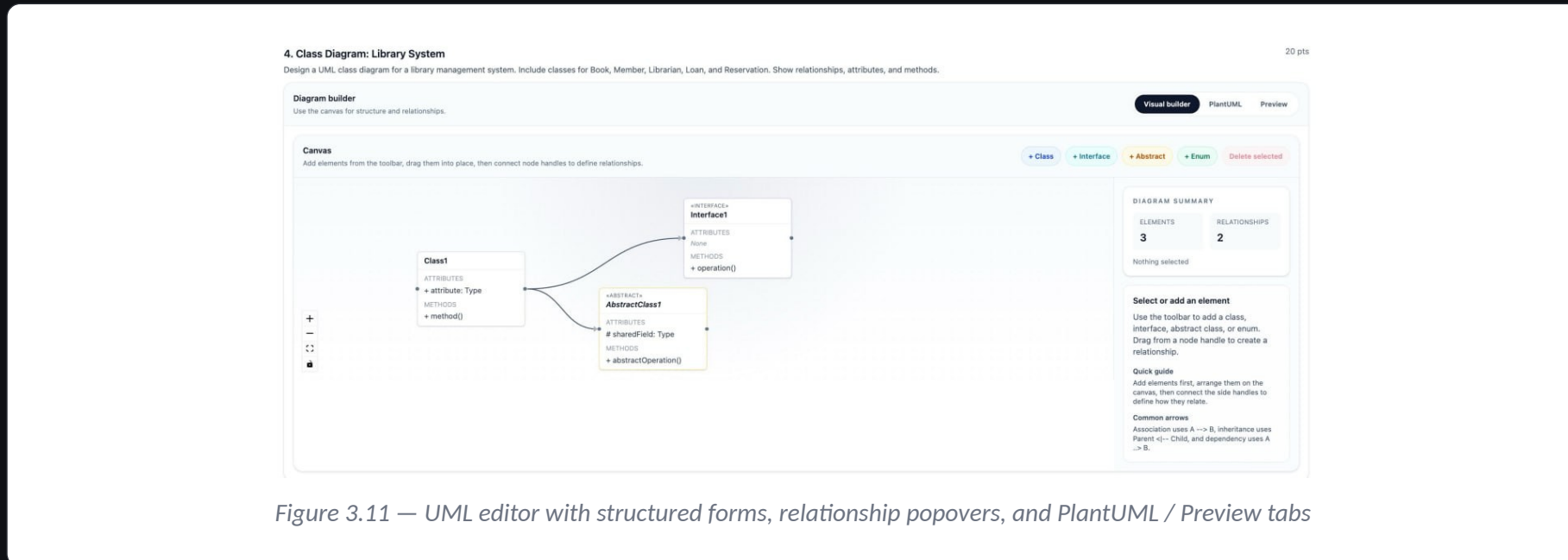


Figure 3.11 — UML editor with structured forms, relationship popovers, and PlantUML / Preview tabs

01 React Flow base

Class + sequence diagrams.
Custom nodes/edges:
associations, inheritance,
lifelines, messages.

02 Structured input

Visibility selectors + typed names
— not freeform UML notation.

03 Two-export design

Serialises to PlantUML DSL and
PNG; both deterministic from
editor state.

04 Component reuse

Same UMLEditor for student
attempts; UMLViewer for staff
grading.

• AI Grading • Async Job Architecture

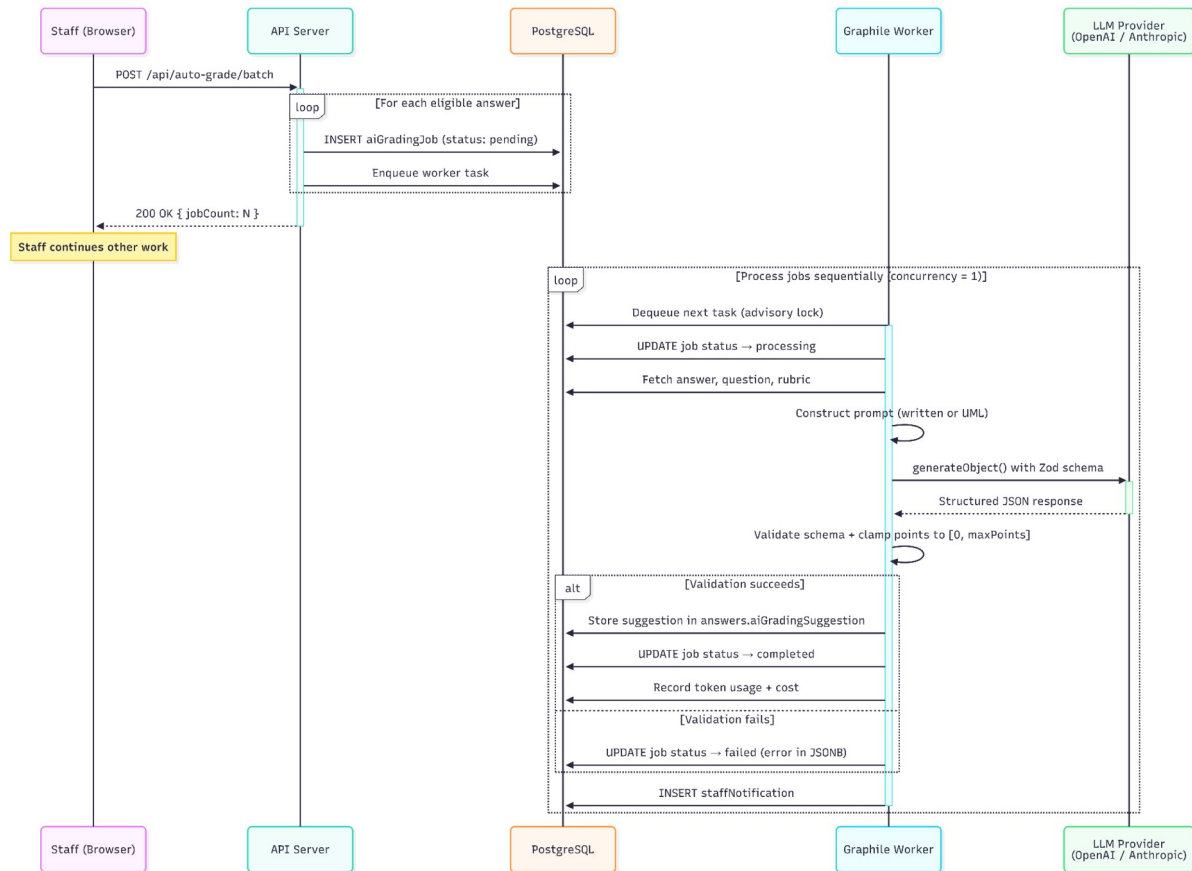


Figure 3.9 — Batch AI grading sequence

01 Fire-and-return

API enqueues N jobs and returns immediately with jobCount.

02 Concurrency = 1

Sequential processing avoids LLM rate-limit contention.

03 Pipeline

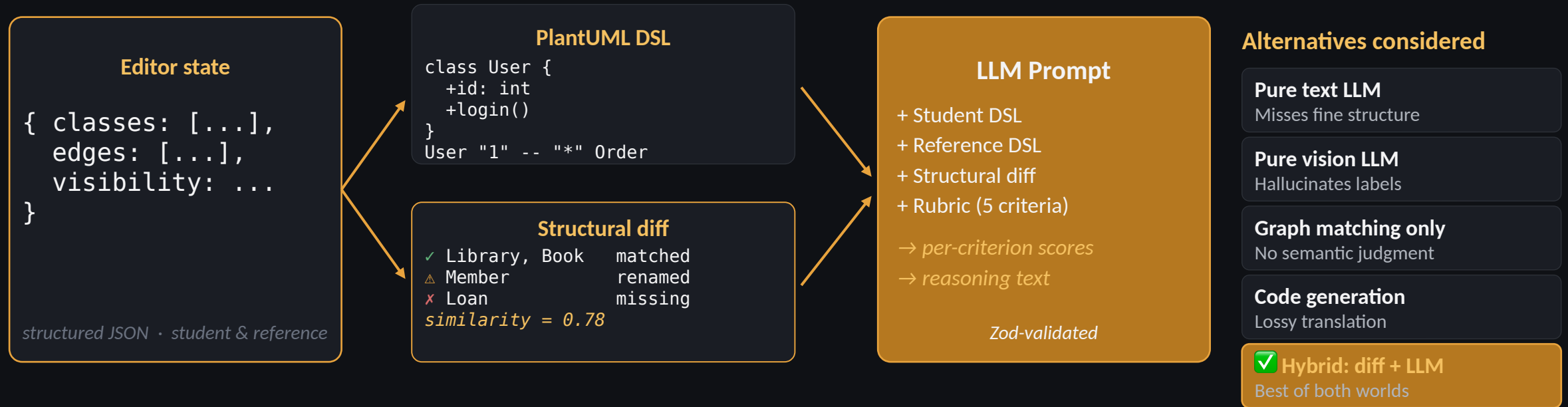
Fetch → prompt → call → validate → store, with token usage logged.

04 Visible failures

Errors recorded with full message + staff notification — never silent.

★ Hybrid Symbolic + Neural Grading

deterministic graph-diff anchors structural correspondence; the LLM grades semantic criteria



Why it works: the diff catches what LLMs are unreliable at — precise structural correspondence — and frees the LLM to focus on naming, design judgment, and partial credit.

• Structured Output & Defensive Validation

schema.ts — Zod schema for grading output

```
const gradingSchema = z.object({
  points: z.number().min(0),
  reasoning: z.string().min(10),
  confidence: z.number().min(0).max(100),
  criteriaScores: z.array(z.object({
    criterion: z.string(),
    score: z.number(),
    comment: z.string(),
  })).optional(),
});

// Symbolic step: deterministic graph-diff
const diff = diffClassDiagrams(student, reference);

// Neural step: structured-output LLM call
const { object } = await generateText({
  model: getModel(settings),
  output: Output.object({ schema: gradingSchema }),
  prompt: buildPrompt(student, reference, diff, rubric),
});

// Defensive clamp
object.points = Math.min(object.points, maxPoints);
```

01 API-level enforcement

Vercel AI SDK `Output.object()` enforces the schema at the provider level — no fragile JSON parsing.

02 Schema validation

Malformed output → job fails explicitly. No corrupt data ever reaches the gradebook.

03 Score clamping

Bounded to `[0, maxPoints]` — guards against out-of-range hallucinations.

04 Result

100% parse rate on Claude Opus and Sonnet (see Section 04).

Human-in-the-Loop Review

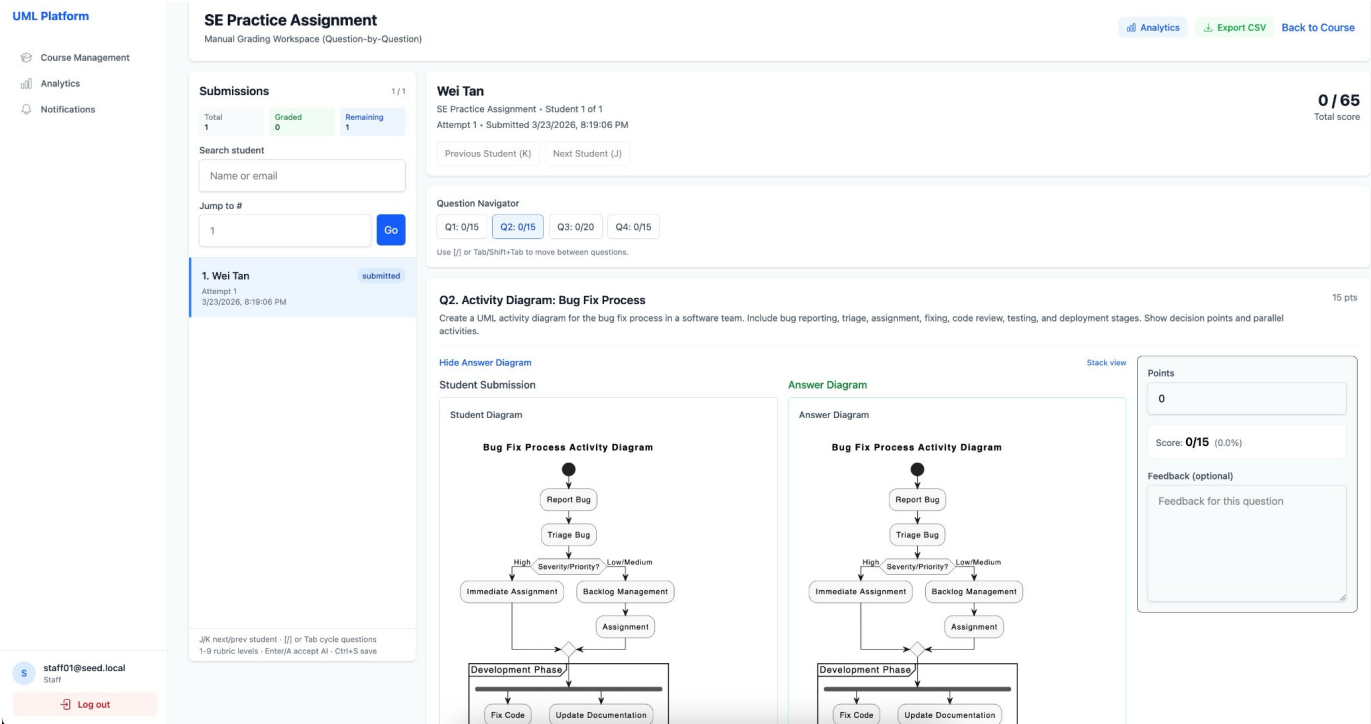


Figure 3.10 — Two-panel grading interface: AI suggestion (left) · official mark (right)

01 Suggestions are staged

AI never writes to gradebook directly — separate from marks table.

02 Accept

Official mark recorded with source = ai_assisted; original suggestion preserved.

03 Reject

Suggestion cleared; staff grades manually.

04 Override

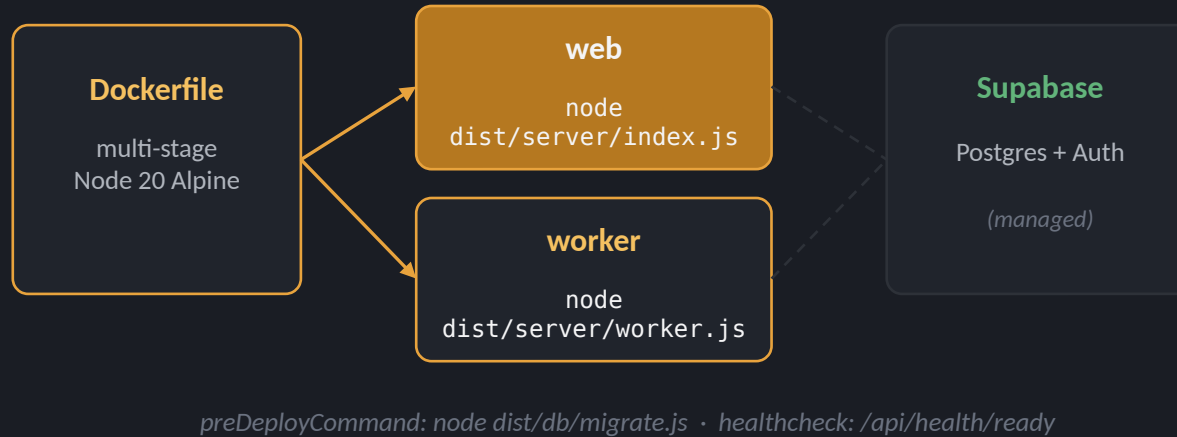
Mark changes recorded with optional written reason and previous-score audit.

05 Auditability

Every published grade has verifiable human oversight.

• Production Deployment & CI/CD

Railway • two services from one Dockerfile



01 Two services, one image

Same Dockerfile, different startCommand for web vs worker.

02 Healthcheck-gated rollout

/api/health/ready returns 200 only when DB is reachable; Railway holds traffic until green.

03 Migration as preDeploy hook

node dist/db/migrate.js runs before the new web pod starts; rollout aborts if migration fails.

04 GitHub Actions pipeline

lint → vitest → build & push image to ghcr.io. Bridge to the on-prem path.

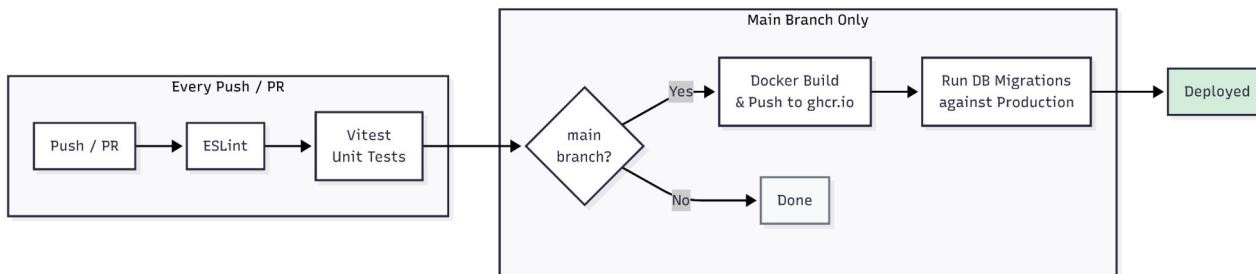


Figure 3.14 — GitHub Actions: lint · test · build · push · migrate

• On-Premise Deployment Path

Why on-prem? Some institutions cannot use managed PaaS — outbound traffic policies, data sovereignty, or pre-existing on-prem clusters. The deployment artefact is reusable on Kubernetes.

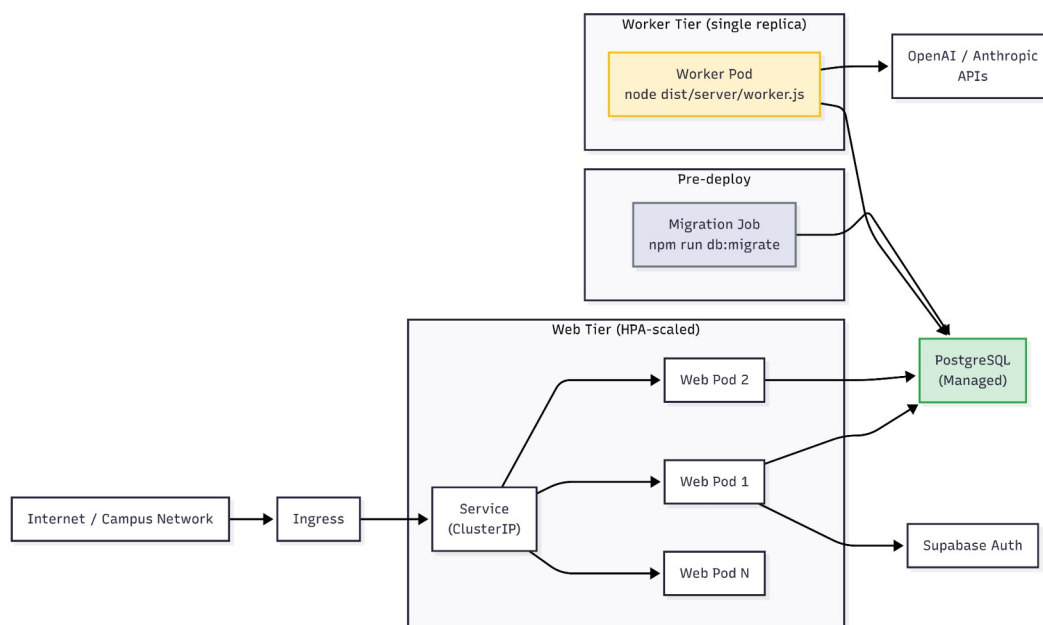


Figure 3.13 — Kubernetes topology: HPA-scaled web tier, single-replica worker, migration Job

01 Same image, k8s manifests

The CI/CD image is what the cluster pulls.

02 Web tier: HPA 2 → 6

CPU-based autoscale for exam bursts.

03 Worker tier: single replica

Advisory locks make scale-out safe if needed.

04 Pre-deploy migration Job

kubectl apply + wait — pre-rollout gate.

Reference architecture; not deployed live. Fully air-gapped deployment (Ollama for LLM, self-hosted Postgres) is future work — see Section 05.

04

SECTION

Experiment & Evaluation



● Research Questions

RQ1

Accuracy

Which LLM achieves the highest grading accuracy compared to human experts?

RQ2

Cost-performance

What is the cost-performance tradeoff between flagship and baseline models across providers?

RQ3

Consistency

How consistent are LLM grades across multiple runs on the same submission?

RQ4

Reliability

Which model produces the most reliable structured output and useful qualitative feedback?

● Experiment Setup

Models • Table 4.1

Model	Provider	Tier	\$/1M in/out
Claude Opus 4.6	Anthropic	Flagship	\$5 / \$25
Claude Sonnet 4.6	Anthropic	Baseline	\$3 / \$15
GPT-5.4	OpenAI	Flagship	\$2.50 / \$15
GPT-5.4 Mini	OpenAI	Baseline	\$0.75 / \$4.50
Gemini 3.1 Pro	Google	Flagship	\$2 / \$12
Gemini 3 Flash	Google	Baseline	\$0.50 / \$3

Dataset & Protocol

32

submissions

6

models

5

runs each

= 960 API calls

Submissions: 2 real (McGill UML repo) + 30 synthetic across 5 quality tiers

Domains: Library Management · E-Commerce · Hospital Management

Rubric: 5 criteria — class correctness · relationships · cardinality · naming · completeness

Protocol: temperature 0.0, exponential backoff, every call logged

5 synthetic-quality tiers

●

Excellent

●

Good

●

Average

●

Poor

●

Failing

● Evaluation Metrics

Metric	Weight	Captures
Accuracy	35%	Pearson r + MAE vs ground truth
Consistency	25%	Std. dev. across 5 runs
Cost	10%	USD per submission
Feedback quality	15%	Parse success + structured-output compliance
Speed	5%	Median latency
Rubric adherence	10%	% rubric criteria addressed

$$W = 0.35 \cdot S_{acc} + 0.25 \cdot S_{con} + 0.10 \cdot S_{cst} + 0.15 \cdot S_{fdb} + 0.05 \cdot S_{spd} + 0.10 \cdot S_{rub}$$

Each sub-score normalised against an absolute reference ceiling (not min-max) — preserves stability when models are added or removed.

★ RQ1 — Grading Accuracy

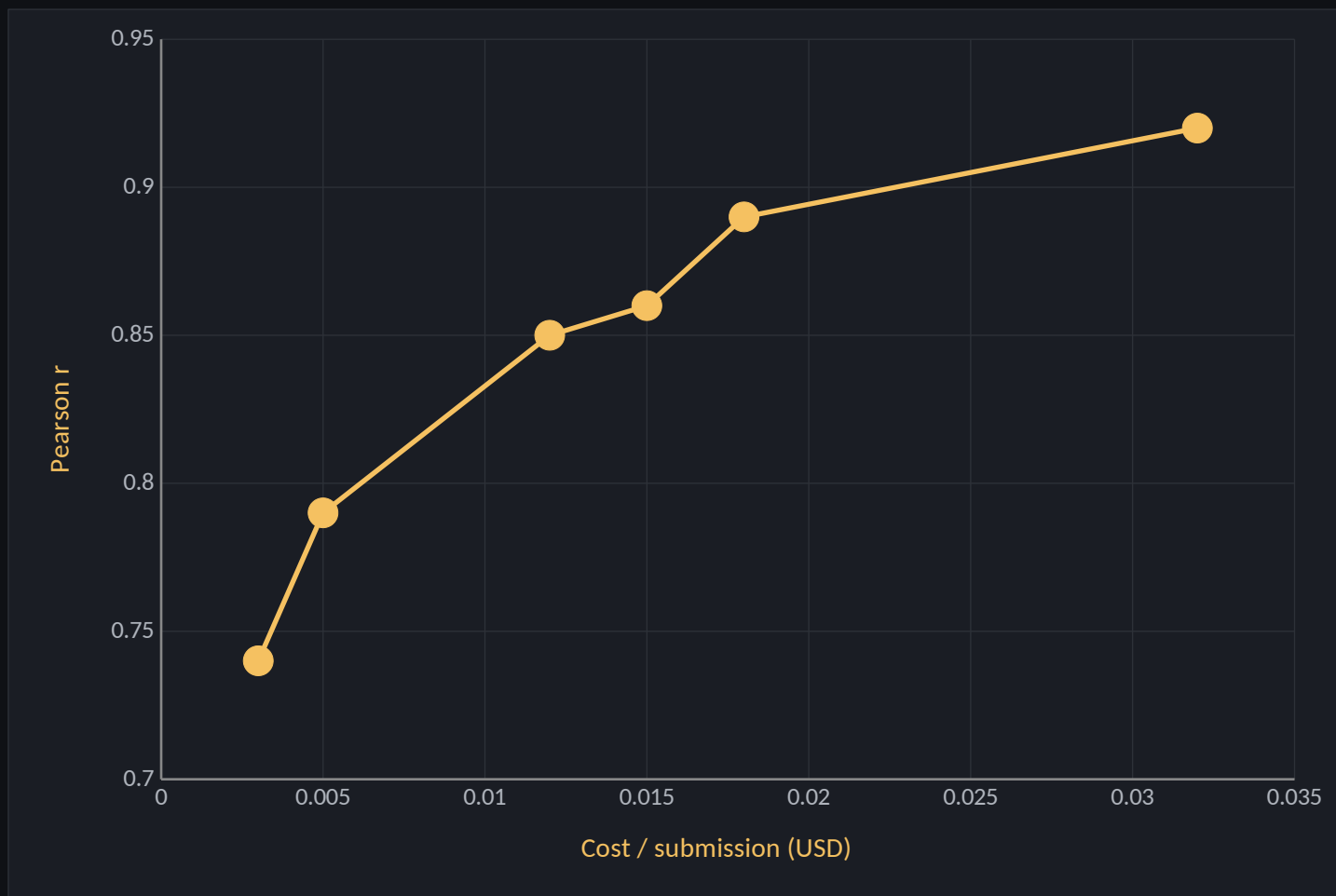
the empirical headline

Model	Pearson r	MAE /10	Mean Bias
Claude Opus 4.6	0.92	0.68	-0.28
Claude Sonnet 4.6	0.89	0.78	-0.42
GPT-5.4	0.86	0.90	-0.55
Gemini 3.1 Pro	0.85	0.95	-0.52
GPT-5.4 Mini	0.79	1.25	-0.80
Gemini 3 Flash	0.74	1.48	-1.02



$r = 0.92$ improves on Bouali et al. (ICC = 0.76, earlier-gen models). All models show negative bias — systematic over-strictness, consistent with prior work.

● RQ2 • Cost-Performance Tradeoff



01 \$0.003 → \$0.032

10× cost span across the lineup, all operationally affordable.

02 200-cohort total

Entire grading bill ranges \$0.60 → \$6.40 per assignment.

03 Sweet spot: Sonnet 4.6

\$0.018 / $r = 0.89$ — half Opus's cost, near-equal accuracy.

04 Within-provider gap is tight

Institutions can choose by budget without sacrificing quality.

● RQ3 • Consistency + RQ4 • Structured Output

RQ3 • Inter-run consistency

Model	Mean σ	Max σ
Claude Opus 4.6	0.16	0.50
Claude Sonnet 4.6	0.21	0.65
Gemini 3 Flash	0.48	1.60

Lower σ = more reliable across re-runs

RQ4 • Structured-output parse rate

100%	Claude Opus / Sonnet
$\geq 97.5\%$	all flagship models
94.4%	Gemini 3 Flash • truncated reasoning

Why both findings matter:

Opus's $\sigma = 0.16$ sits well below the 1.5-point ceiling that would mean two students could receive grades a full letter apart on the same submission. Combined with 100% parse rate, the API-level structured-output enforcement and temperature 0.0 protocol have effectively resolved the format-compliance problem flagged by *Piscitelli et al. (2025)*.

● Aggregate Ranking & Recommendation

Rank	Model	Weighted Score	Recommendation
1	Claude Opus 4.6	0.87	Default for production deployment
2	Claude Sonnet 4.6	0.87	Cost-effective alternative
3	GPT-5.4	0.84	Fallback when Anthropic unavailable
4	Gemini 3.1 Pro	0.83	Not recommended
5	GPT-5.4 Mini	0.80	Not recommended
6	Gemini 3 Flash	0.77	Not recommended

Tie at 0.87 — Opus leads by < 0.01 at full precision (driven by accuracy + consistency). Opus = production default; Sonnet = cost-effective alternative; GPT-5.4 = viable fallback.

• Threats to Validity

01 Synthetic-heavy dataset

Only 2 of 32 submissions are real student work.

02 Single-grader ground truth

Most significant threat — no inter-rater reliability measured.

03 Experiment scope

Empirical study covered class only. Sequence now supported in editor but unbenchmarked.

04 Conversion artefact

McGill submissions required Umple → PlantUML conversion.

05 Model drift

Providers update models over time; results are a snapshot.

Future work prioritises real student data with multi-grader ground truth — see Section 05.

05

SECTION

Conclusion



• Summary & Future Work

Delivered

- ✓ Full-stack platform with end-to-end assessment lifecycle [O1]
- ✓ Hybrid symbolic + neural grading (graph-diff + LLM); 100% parse rate on Claude [O2]
- ✓ Empirical: Claude Opus 4.6 wins ($r = 0.92$), Sonnet best cost-quality [O4]
- ◆ Two-panel grading UX — exact time saving pending user study [O3]
- ◆ Containerised + k8s deployment — load-test under realistic exam load pending [O5]

Future Work

01 More diagram types

Class + sequence shipped. Extend editor to activity, state, ER. Pipeline already type-agnostic.

02 Real-student benchmark

Organic student work across multiple courses. Closes the Bouali et al. standardisation gap.

03 Multi-grader validation

3+ graders, ICC metrics, recalibrate the $-0.28 \rightarrow -1.02$ LLM bias estimates.

04 Air-gapped deployment

Ollama / vLLM for on-prem LLM serving; self-hosted PostgreSQL StatefulSet.

Thank you

Q & A

Wang Yu Teng (U2122796L) · CCDS25-0233

Supervisor: Dr. Loke Yuan Ren · Examiner: Prof. Zhang Hanwang

College of Computing and Data Science · 2026